

Ruben van der Vyver 25007786

Vimbai Chivunga u25136608

Tafadzwa Manyengawana u25370325

Task 1

Component -> WorkElement

Composite -> WorkGroup

Leaf -> ResponseUnit and other subclasses

Context -> ResponseUnit

State -> UnitState

ConcreteState -> StagedState and attached

Aggregate -> WorkGroup

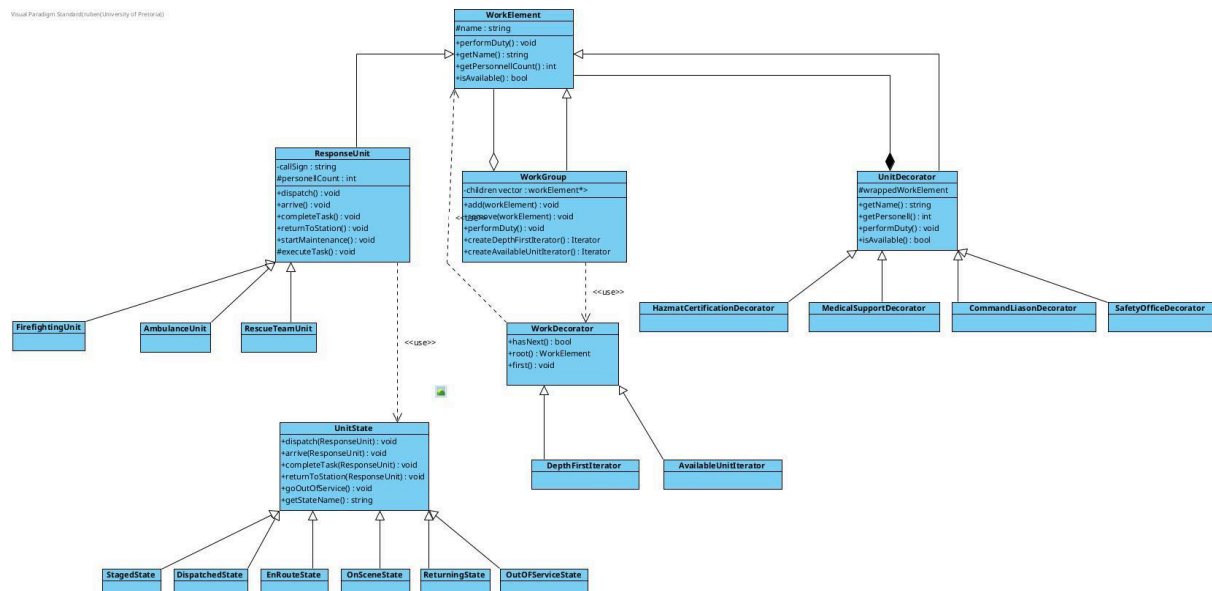
Iterator -> WorkIterator

ConcreteIterator -> DepthFirstIterator and the like

Decorator -> UnitDecorator

Component -> WorkElement

ConcreteDecorator -> HazmatCertificationDecorator and such



Design rationale:

Task 3

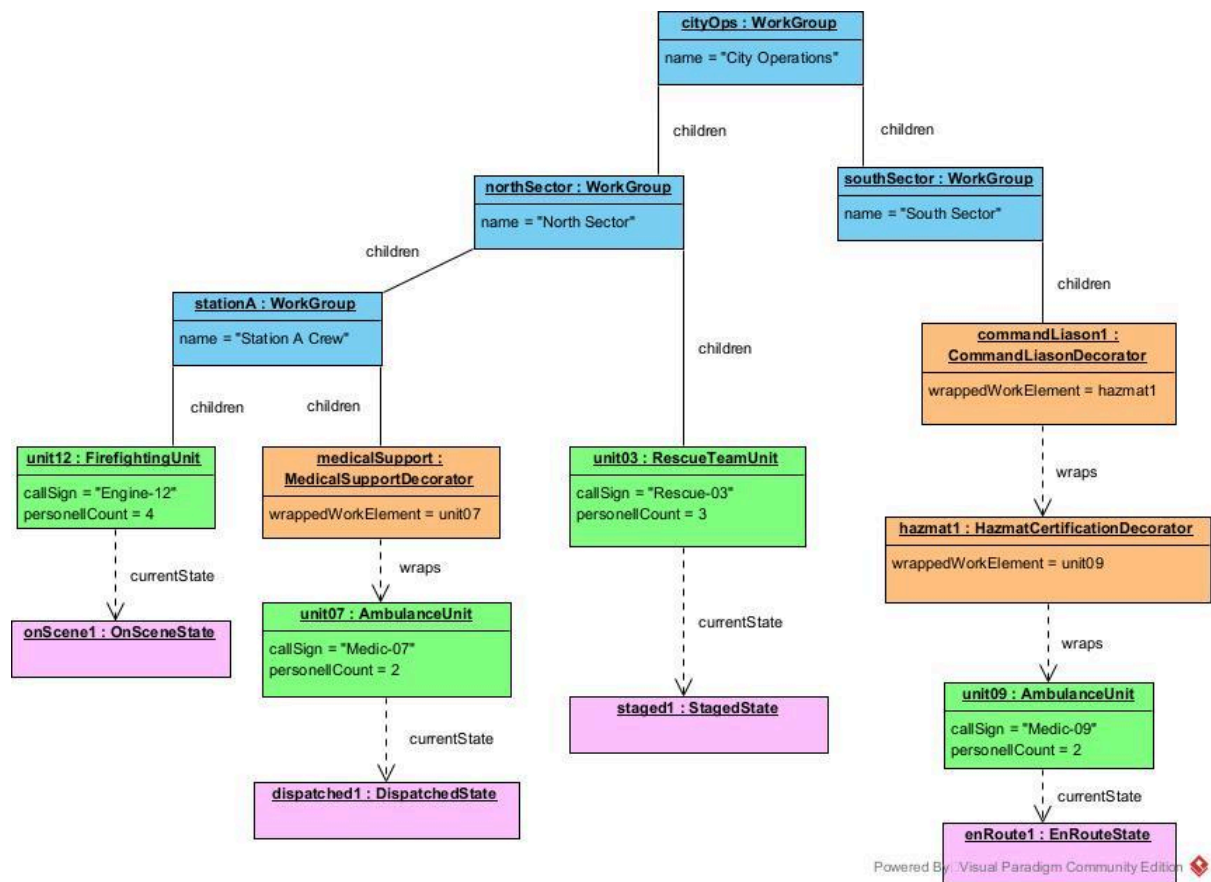
TaskForge shows its dynamic behaviour through two scenarios built on the same incident hierarchy (Incident, Branch, Division, Strike Team, Unit, nested in that order). In the first scenario, an iterator walks the whole hierarchy and prints a roll call without ever reaching into a group's internals directly. Engine 21 is then moved through its lifecycle step by step: dispatched, en route, on scene. When we try to dispatch it again while it's already on scene, the system rejects the action and logs it instead of crashing. This shows traversal and state based behaviour working on the same object.

In the second scenario, two iterators run at the same time and don't interfere with each other: one lists every unit, the other lists only available ones. Engine 22 is taken out of its team, wrapped with two decorators (medical support and hazmat), and put back. It then carries out its duty through both decorators just like a normal unit would, showing decoration working as part of normal use, not just as a demo. Rescue 5 is also moved from Division A into Strike Team 1 while keeping its current state, which is our runtime structural change. Both iterators work by taking a snapshot of the hierarchy the moment they're created, so later changes (like moving Rescue 5) don't affect an iterator that's already running. We prove this by showing that Rescue 5's move doesn't disturb either of the two running iterators. The one rule this depends on is that units can be moved or decorated while a traversal is happening, but not deleted. Deleting one mid traversal would leave the iterator holding a pointer to something that no longer exists. We tested this directly by deleting a unit while an iterator was still using it, and saw it crash under GDB and Valgrind (write up in Task 5). We also checked that TaskForge itself never does this: every deletion only happens at the very end, once every iterator has already finished and been destroyed.

Task 4

UML Object Diagram:

Captures a runtime snapshot of the ResponseSector composite hierarchy, displaying explicit node ownership, active UnitState instances, and dynamically stacked UnitDecorator wrappers.

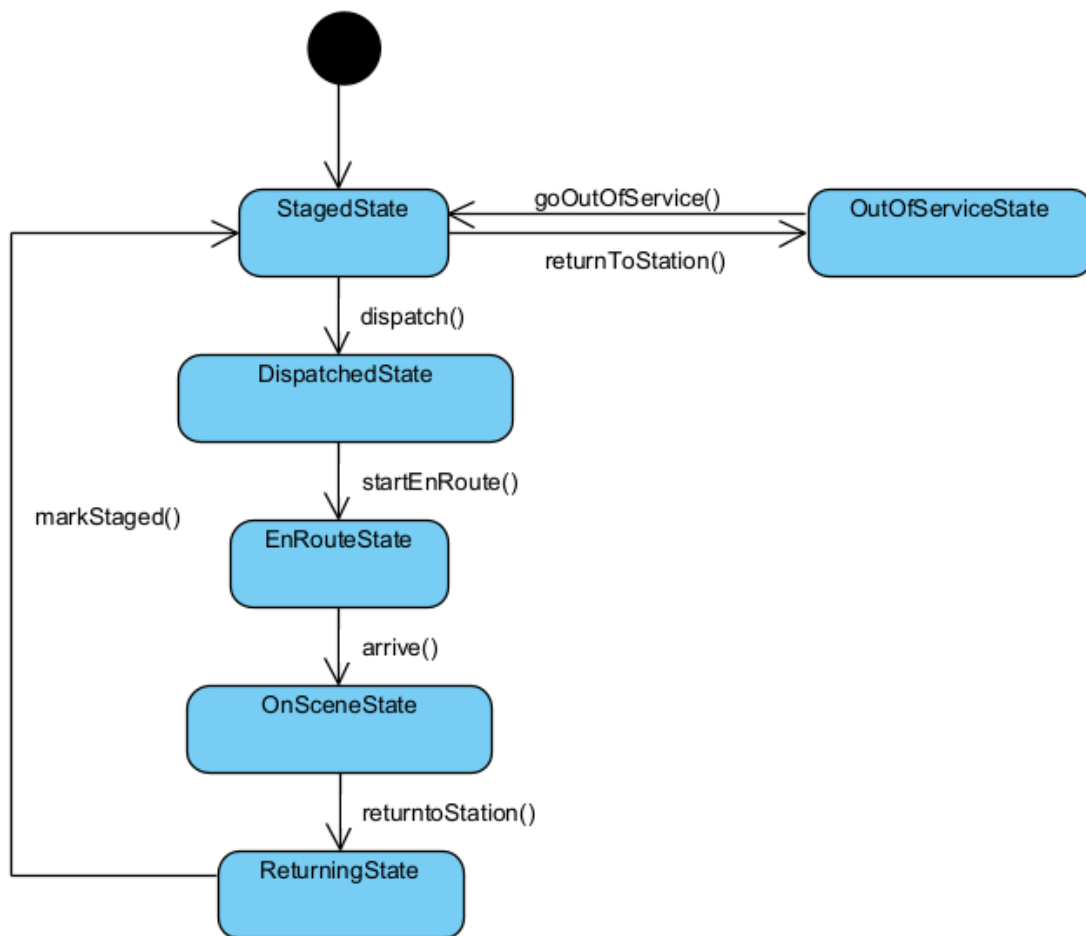


UML State Diagram:

Models the operational lifecycle of a stateful ResponseUnit.

Lifecycle Transitions:

- StagedState: Initial state upon instantiation where the unit awaits operational assignment.
- DispatchedState: Entered when dispatch() is invoked on ResponseUnit. The unit transitions from StagedState to DispatchedState and begins transit.
- OnSceneState: Entered when arrive() is invoked. The unit executes active operational duties via performDuty().
- CompletedState: Entered when complete() is called upon task completion, releasing the unit back to standby.

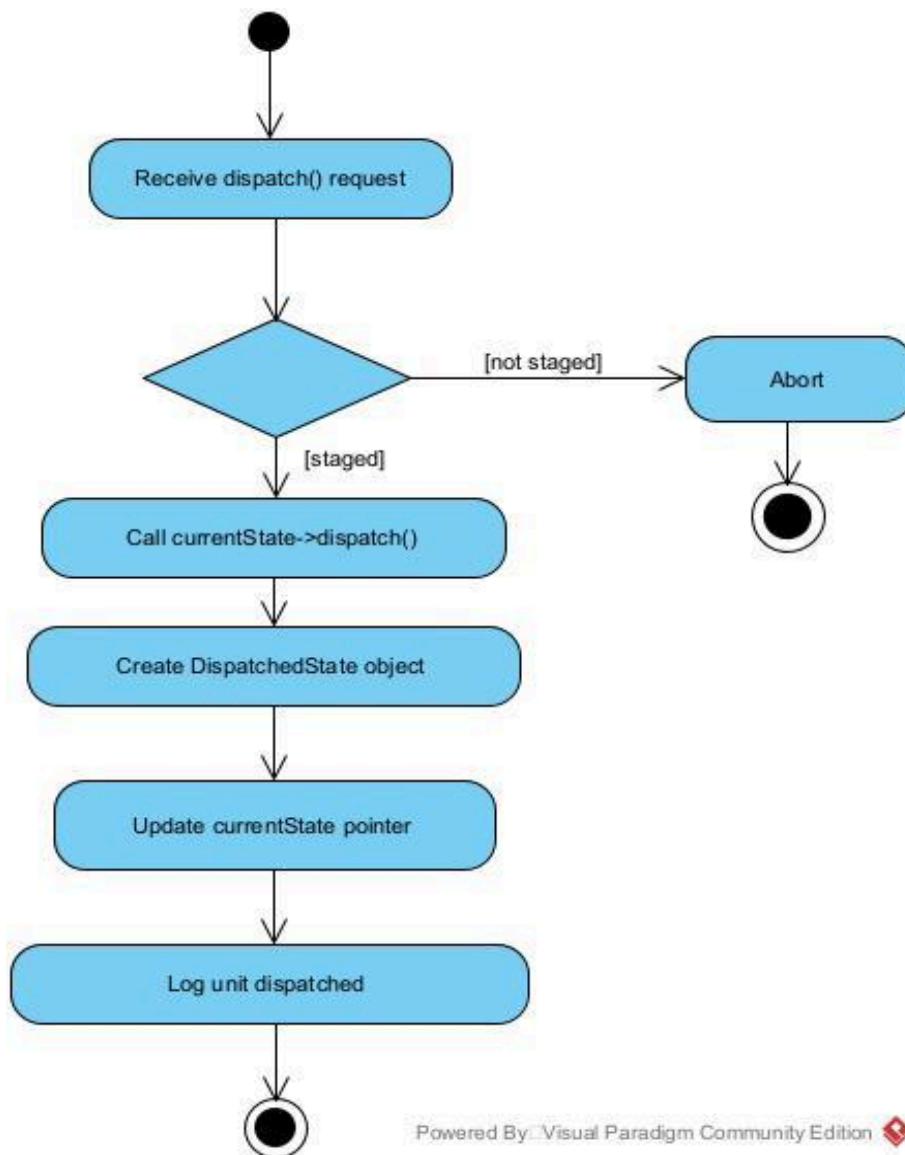


Activity Diagram 1: Sector-Wide Dispatch Workflow

Demonstrates sector-wide dispatching by making composite structure traversal explicitly visible step-by-step rather than hiding it behind a single generic action block.

UML Construct Compliance: Features decision diamonds, explicit loop edges, merge nodes, and guard conditions `[hasMore == true]` and `[hasMore == false]`.

Code Traceability: Maps directly to `Unitlterator::first()`, `Unitlterator::hasMore()`, `Unitlterator::next()`, and `ResponseSector::dispatch()`.

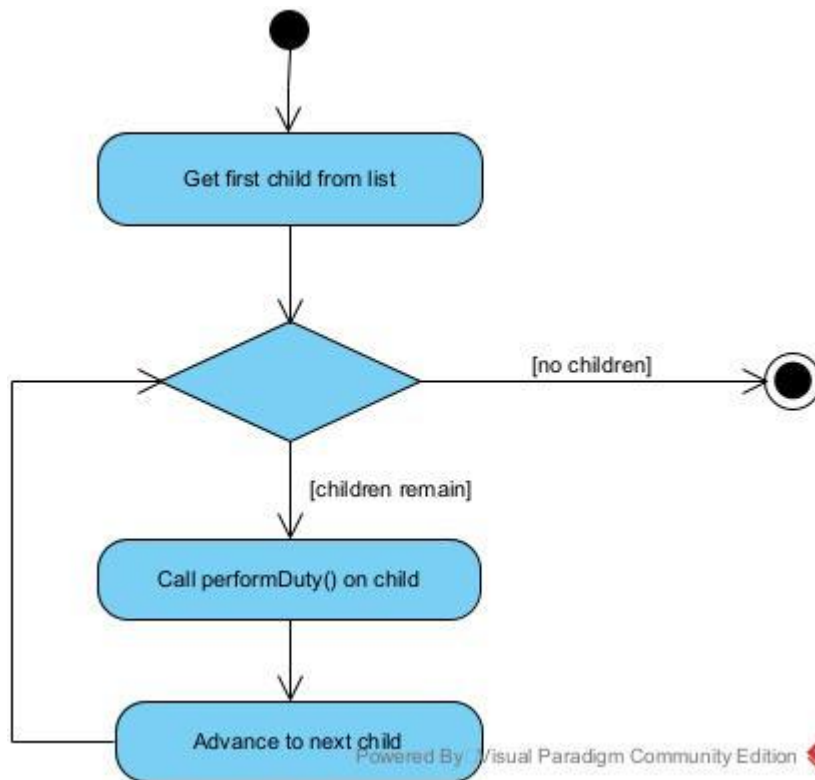


Activity Diagram 2: Execute Duty Workflow

Models the execution of duties across different organizational tiers, highlighting state-dependent conditional branching and concurrent execution.

UML Construct Compliance: Structured using three swimlanes (ResponseSector, FieldUnit, UnitState), decision diamonds with state guards ([currentState == Staged], [currentState == Dispatched], [currentState == OnScene]), and parallel fork/join synchronization bars.

Code Traceability: Maps to ResponseUnit::performDuty(), delegating execution through ResponseUnit::currentState->performDuty() and triggering state transitions via transitionTo().

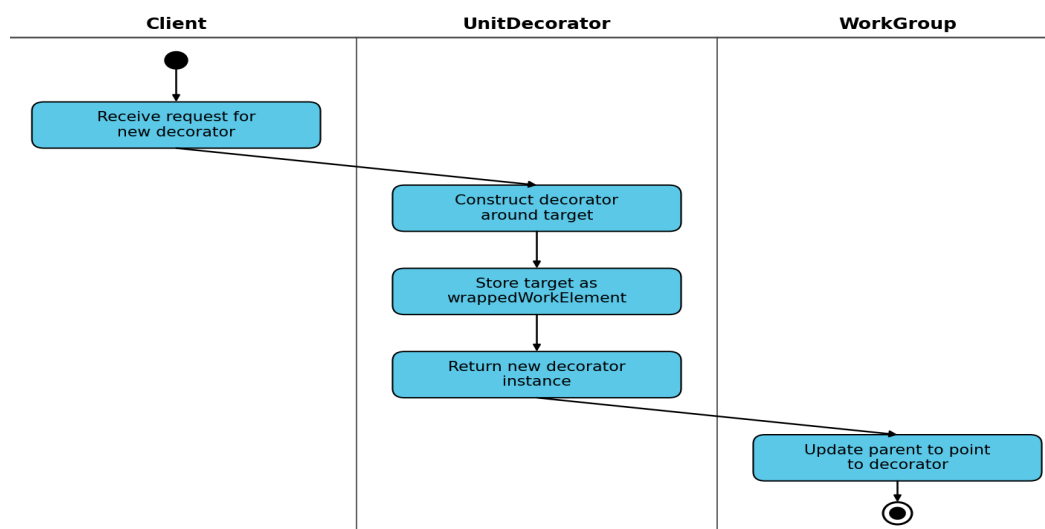


Activity Diagram 3: Apply Decorator Workflow

Models the dynamic runtime wrapping of a target work element with additional operational responsibilities.

UML Construct Compliance: Features an initial node, single-phrase action nodes, a composite activity invocation block, and an activity final node.

Code Traceability: Maps directly to `UnitDecorator::UnitDecorator(ResponseUnit* target)`, setting `wrappedWorkElement = target`, and updating the parent `ResponseSector` pointer via `ResponseSector::replaceChild()`.



Task 5

1 Bug found

While testing the iterator, I found a bug where the program would crash if a unit was deleted while the iterator was still being used. The iterator still had a pointer to the deleted unit, so when it tried to access it again, it was accessing invalid memory. I used GDB to step through the code and check the iterator and unit pointers, which helped me find that the iterator was still pointing to the deleted object. Instead of changing the whole iterator to handle deletions, I decided that units should not be deleted while an iterator is currently being used. The main program already follows this rule, so this should not happen during normal use. I also tested what happens when the rule is broken, which confirmed that the crash was caused by the invalid pointer.

I ended up deciding on a policy that units should not be deleted while an iterator is being used to traverse them.

GDB Debug

Testing case against policy (Deleting a unit when an iterator is going to use them)

```
section("Iterator invalidation test");
WorkIterator* fullWalk = incident->createDepthFirstIterator();
delete opsBranch;
std::cout << "Continuing traversal after deletion:\n";
while (fullWalk->hasNext()) {
    std::cout << fullWalk->next()->getName() << "\n";
}
delete fullWalk;
std::cout << "Units currently available:\n";
while (availableWalk->hasNext()) {
    std::cout << " " << availableWalk->next()->getName() << "\n";
}
delete availableWalk;
```

```

Program received signal SIGSEGV, Segmentation fault.
0x000055555555a458 in main () at main.cpp:72
72      std::cout << fullWalk->next()->getName() << "\n";
(gdb) bt
#0 0x000055555555a458 in main () at main.cpp:72
(gdb) list
67      section("Iterator invalidation test");
68      WorkIterator* fullWalk = incident->createDepthFirstIterator();
69      delete opsBranch;
70      std::cout << "Continuing traversal after deletion:\n";
71      while (fullWalk->hasNext()) {
72          std::cout << fullWalk->next()->getName() << "\n";
73      }
74      delete fullWalk;
75      std::cout << "Units currently available:\n";
76      while (availableWalk->hasNext()) {
(gdb) info locals
incident = 0x555555576ee0
opsBranch = 0x555555576f80
medBranch = 0x555555576ff0
divisionA = 0x555555577040
strikeTeam1 = 0x555555577090
engine21 = 0x5555555770e0
engine22 = 0x555555577120
rescue5 = 0x555555577160
ambulance9 = 0x5555555771a0
rollCall = 0x555555576eb0
availableWalk = 0x555555576eb0
fullWalk = 0x555555577800
decoratedEngine22 = 0x7ffff7e28e88 <std::wclog+8>
(gdb)

```

Valgrind output (no memory leaks found)

```

==9348== Memcheck, a memory error detector
==9348== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==9348== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==9348== Command: ./taskforge
==9348==

```

----- Roll call (DepthFirstIterator - complete traversal) -----

Incident 47 - Warehouse Fire

Operations Branch

Division A

Strike Team 1

Engine 21

Engine 22

Rescue 5

Medical Branch

Ambulance 9

----- Lifecycle: Engine 21 responds -----

Engine 21 dispatched.

Engine 21 confirms en route.

Engine 21 arrived on scene.

[INVALID] Engine 21 cannot dispatch while OnScene.

----- Two independent iterators over the same hierarchy -----

Units currently available:

Engine 22

Rescue 5

Ambulance 9

Full roll call, first 3 entries, unaffected by the above:

Incident 47 - Warehouse Fire

Operations Branch

Division A

----- Dynamic decoration: Engine 22 gets hazmat + medical support -----

Engine 22 [MEDIC] [HAZMAT] now carries 6 personnel.

Engine 22 [Staged]: laying hose lines and applying water.

+ medical support: providing on-scene casualty care.

+ hazmat certification: monitoring air quality and decontamination.

----- Re-tasking: Rescue 5 moves from Division A into Strike Team 1 -----

Rescue 5 dispatched.

Rescue 5 confirms en route.

----- Full duty performance after all changes -----

Incident 47 - Warehouse Fire

Operations Branch

Division A

Strike Team 1

Engine 21 [OnScene]: laying hose lines and applying water.

Engine 22 [Staged]: laying hose lines and applying water.

+ medical support: providing on-scene casualty care.

+ hazmat certification: monitoring air quality and decontamination.

Rescue 5 [EnRoute]: conducting search and technical rescue.

Medical Branch

Ambulance 9 [Staged]: triaging and transporting patients.

----- Shutdown -----

Incident structure torn down cleanly.

==9348==

==9348== HEAP SUMMARY:

==9348== in use at exit: 0 bytes in 0 blocks

==9348== total heap usage: 58 allocs, 58 frees, 75,857 bytes allocated

==9348==

==9348== All heap blocks were freed -- no leaks are possible

==9348==

==9348== For lists of detected and suppressed errors, rerun with: -s

==9348== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

Task 6 (Readme)

Running

Setting up docker environment

docker build -t taskforge .

docker run --rm -it taskforge bash

Building and running program

make

./taskforge

Debugging

gdb ./taskforge

valgrind --leak-check=full --show-leak-kinds=all ./taskforge